Lecture 12: Designing Instagram feed / Idempotent APIs

Q: What are the features that we need to design?

↳ Create Post
follow / Unfollow
see his/her own feed

Q: How many users can we follow?

↳ Unlimited

Q: Daily Active Users?

↳ 20 million

→ Back of the Envelope calculations:

QPs
Storage Unit

DAU: 20 million
Assumptions:
QPs
Read → Feed read (1 req)
Write → post (1 req)
user refresh/opens his/her own feed → 10 times a day
user post → 1 picture a day (1 MB)

Write: Read → 1:10
Storage unit: (Assuming for 5 years)

QPs → Read QPs, Write QPs?
Storage Unit?

- Functional Requirements :
  1. User can create a post (img/video/reel)
  2. User can follow another user
  3. User can see his / her own feed

- Non-functional Requirement :
  1. Availability is more important then consistency
  2. Scalable (It could handle Billions of users)

Social media interaction :

Facebook , Instagram , LinkedIn

Users → U2 → U3

GraphDB → Neo4J (No-SQL)

facebook : friend → Bi-directional connection

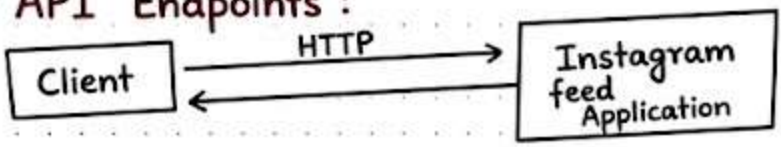
U1 ⟷ U2 (Undirected graph)

Instagram : follow → Uni-directional connection

U1 ---> U2 (Directed graph)

LinkedIn : Hybrid Approach (connection / follow)

↳ API Endpoints :



1. posting something (user → img/video/reel)

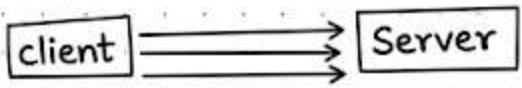
post /create/post/{userId}

Body : {
Caption,
content (img / video / reel),
timestamp
}

'Response : 200 ok

2. following a user (Directed Graph)

Idempotent APIs → Dedupe (double request)
i.e. even if double request comes, our api does not do anything wrong.



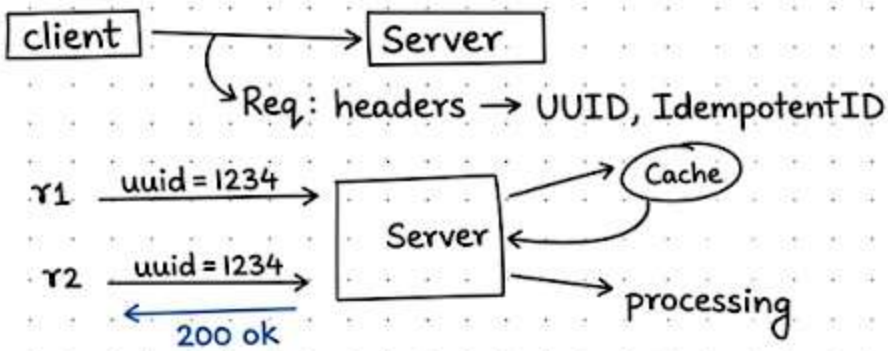
post :

/create/post/{userID}
/create/post/{userID}
Request signature is identical

API → double calls → treat as one

(Idempotent APIs)

Idempotent API



GET, PUT, POST, DELETE

patch

- Get, PUT, and DELETE are Idempotent by default.

2. follow a user (Idempotent by default)

U1 ⟶ U2 (follow) → Graph DB → Neo4J

put : /user/{userId}

{

}

Response : 200 ok

3. View our feed

GET : fetch/feed/{userId}

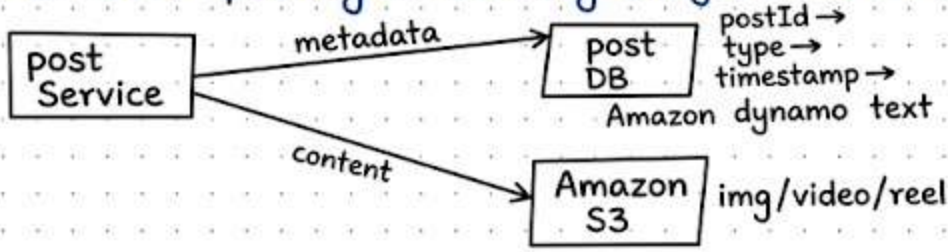
Response :

{ [postId, timestamp, postName, content

}. [], [], [] }

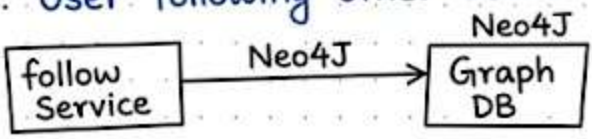
↳ High level Overview :

1. User posting something (img|video|reel)



- 1. Used simple key-value store to store our metadata.
- 2. Used Amazon S3 to store content.

2. User following other users



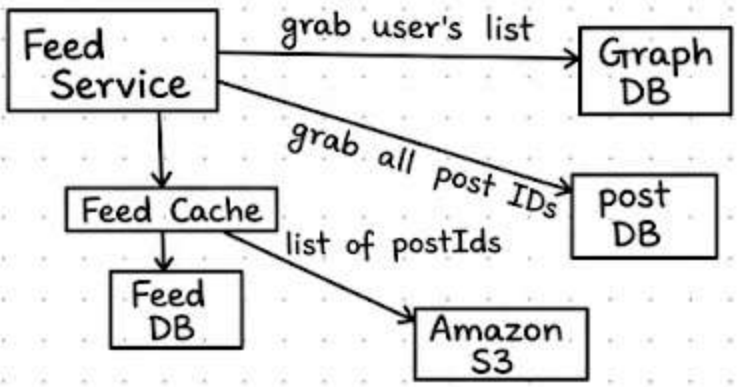
- 1. Many to Many relationships.
- 2. Can be easily model by graph.
- 3. That's why we are using graph DB.

3. User Viewing his/her own feed.

user → post (to whom you follow)
→ Reverse chronological order.

Flow:

1. fetch the user's following list. (metadata)
2. fetch all the new post from this list.
3. finally, sort the the list in reverse chronological order.



Instagram → Read (far more than write.)

Data fetching should be fast.

Write → can be slower but,
Read → should be faster

→ 2 types of model:

- pull Based
- pushed Based

pull Based Model - (Read slow)
(which we made above)

Cons: slow

Pros: Better for users that are inactive

Push-based Model: (Read is fast)

user → post → pre compute in my follower's feed cache.

also called as pre calculation / pre compute model.

Pros: Read is faster

Cons: ↳ Bad for inactive user.

↳ Hot-key problem / celebrity problem

Virat kohli → Instagram account (250 million followers)

Virat → post

For Now, Let's use push based approach:

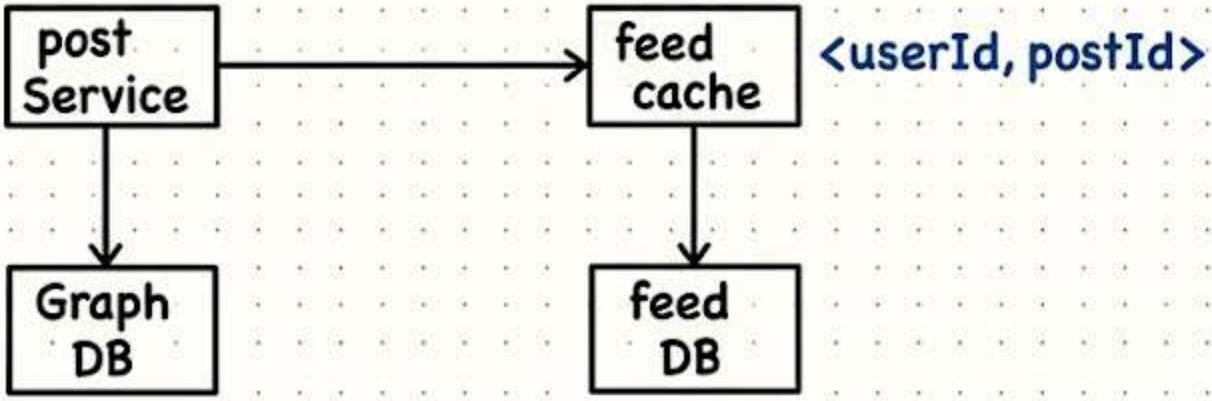
picture post

feed retrive.

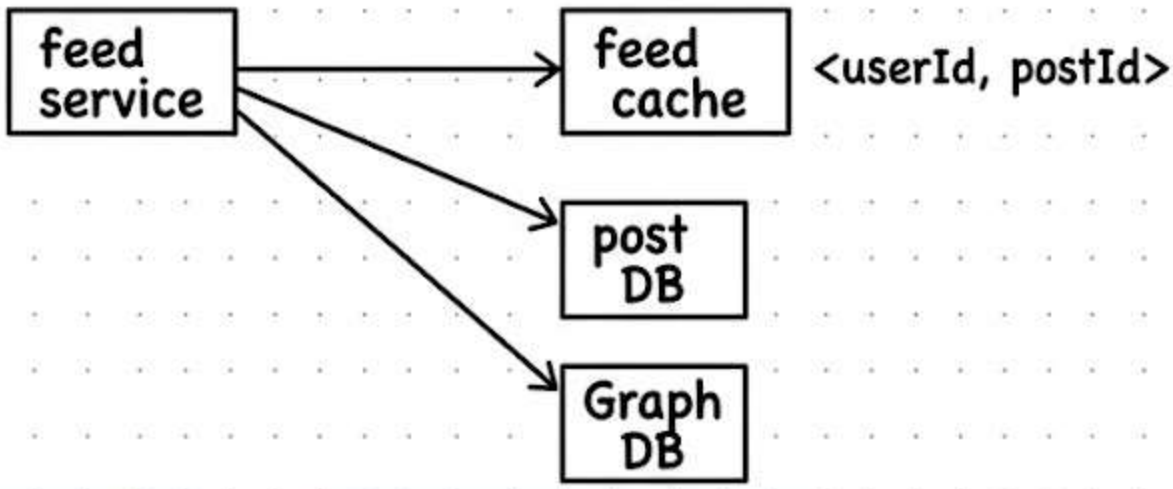
1. User post something

flow:

1. We will fetch the followers list of the current user.
2. then we will add the postId, to the feed cache.
3. We will store a list of pairs of <userId, postId>.



2. When user see his/her own feed



- Everything is pre-computed here

Problems:

1. Both approaches have some disadvantages.
2. Hybrid approach
   ↳ Every user has a boolean variable → isPreComputed
   - people with more followers → pull based approach
   - people with less followers → push based approach

   (follower ≤ 1 million) (less followers → push based approach)

   We can limit the number of posts that appear in feed cache.
   - 200 people data
   - fetch next 200 → pagination